

LangChain: 构建下一代AI智能体的核心引擎

从快速原型到企业级部署，连接大模型与现实世界的桥梁

适用环境与技术栈

类别	具体配置	说明
开发工具	Trae CN	轻量免费、内置终端/代码编辑，完美适配 Windows 系统
运行环境	Python 3.9+	与 LangChain 及 Trae CN 兼容性最佳
核心框架	LangChain 0.1.x（稳定版）	最新稳定版本，功能完善，文档齐全
辅助依赖	chromadb, sentence-transformers, python-dotenv	实现向量存储、嵌入转换等功能
操作系统	Windows 10/11 (x86_64/arm64)	全程适配 Windows 路径、编码等特性

环境准备：配置终端与安装依赖



1. 新建终端会话

点击「终端」→「新建终端」，确保自动关联到项目根目录。



2. 验证环境版本

执行命令检查 Python 与 pip 是否正确安装。



3. 升级 pip 工具

确保使用最新版本的包管理工具以避免兼容性问题。



4. 安装核心依赖

安装 LangChain、ChromaDB 等核心库。

bash — 80x24

```
$python --version
```

```
$pip --version
```

```
$pip install -U pip
```

```
$pip install langchain chromadb \
sentence-transformers python-dotenv \
--upgrade
```


LangChain核心组件（一）



 文档处理

Loader & Splitter
加载与分割文档

 嵌入模型

Embeddings
文本向量化转换

 向量存储

Vector Stores
高效存储向量数据

 检索组件

Retrievers
召回相关知识片段

 LLM 模型

Large Language
Model
生成最终回答

 链组件

Chains
串联全流程逻辑

文档处理 (Loader & Splitter)



Loader (加载器)

核心作用：负责加载各类格式的文档，如 TXT、PDF、Word、CSV 等。

使用：`TextLoader` (文本加载器)



Splitter (分割器)

核心作用：将长文档分割为符合 LLM 上下文窗口限制的小片段 (Chunk)。

使用：`RecursiveCharacterTextSplitter`

文档处理：

01. 准备测试文件

创建 `test\_doc.txt` 文件并写入测试文本，作为后续处理的数据源。

02. 导入必要依赖

```
from langchain.document_loaders import TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

03. 加载与分割文档

```
loader = TextLoader("test_doc.txt", encoding='utf-8')
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50)
split_docs = text_splitter.split_documents(loader.load())
```

04. 验证分割结果

```
print(f"片段数量: {len(split_docs)}")
print(f"第一个片段: {split_docs[0].page_content}")
```

嵌入模型 (Embeddings)



核心作用：文本转向量

将人类可读的文本转换为计算机可理解的嵌入向量 (Embedding Vectors)。



技术基础：语义检索核心

赋予计算机理解文本含义的能力，通过计算向量距离实现相似度比较。



模型选择：SentenceTransformer

采用免费开源模型，无需API密钥，适合快速构建原型与学习。



嵌入模型性能对比：主流闭源模型



● OpenAI (text-embedding-3)

● Cohere (embed-english-v3)

● Anthropic (Claude)

📊 OpenAI: 性能标杆

语义相似度与检索效果最佳，适合追求极致性能的场景，但成本较高。

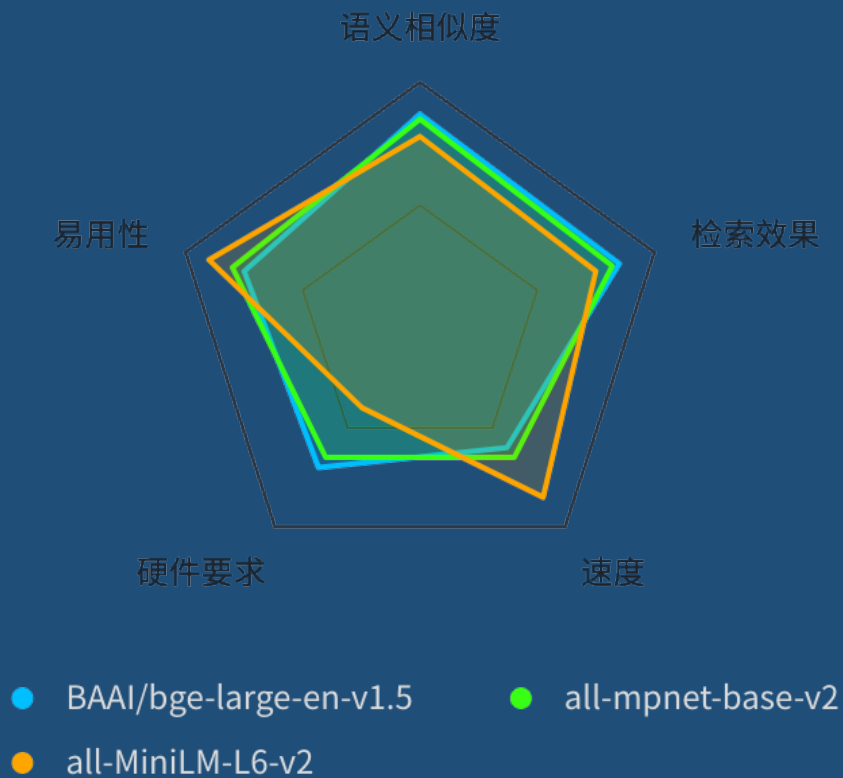
🕒 Cohere: 速度优势

在处理速度上表现突出，综合性能均衡，API集成体验良好。

⚖️ Anthropic: 平衡之选

在性能与成本之间取得了较好的平衡，适合对预算敏感的中型项目。

嵌入模型性能对比：主流开源模型



BAAI/bge-large-en-v1.5 - 高精度首选

在语义相似度和检索效果上表现最佳，但对硬件要求较高，适合追求极致精度的后端服务。

all-mpnet-base-v2 - 均衡之选

各项指标表现均衡，没有明显短板，是通用性强的经典模型，适合大多数常规业务场景。

all-MiniLM-L6-v2 - 极速轻量

推理速度极快且硬件要求极低，适合边缘设备部署或快速原型开发，易用性最高。

嵌入模型

01 导入依赖库

```
from langchain_community.embeddings import SentenceTransformerEmbeddings
```

02 初始化模型

```
embeddings = SentenceTransformerEmbeddings(  
    model_name="BAAI/bge-small-zh-v1.5"  
)
```

03 向量化测试

```
# 单文本向量化  
vec = embeddings.embed_query("入职材料")  
# 多文本向量化  
vecs = embeddings.embed_documents(["入职材料", "离职材料", "工资政策"])
```

向量存储 (Chroma)

核心作用

负责存储文档片段对应的嵌入向量、ID及元数据，并提供高效的向量相似度检索能力。

Chroma

LangChain官方推荐的轻量级向量数据库，支持本地持久化存储，适合开发测试。

核心优势

与LangChain无缝集成，无需复杂配置即可开箱即用，极大降低了开发门槛。



主流向量数据库核心指标横向对比

数据库	类型	核心优势	性能 (P99/QPS)	部署复杂度	成本模型	适用规模
Milvus	开源分布式	高性能、分布式架构	28ms / 3500	高 (需K8s)	按节点计费	十亿~千亿
Pinecone	商业托管	全托管、Serverless	50ms / 1200	 极低	按存储/查询	百万~百亿
FAISS	开源算法库	极致性能、GPU加速	7ms / 极高	 极高	免费(自建)	千万级以下
Weaviate	开源+商业	混合搜索、知识图谱	45ms / 2900	 中	按数据量	百万~十亿
Qdrant	开源高性能	低延迟、内存效率高	22ms / 4200	中	免费/托管	百万~十亿
Chroma	开源轻量级	开发友好、快速原型	25ms / 中	 极低	开源免费	百万级

选型决策树与场景适配建议

核心定位总结

-  **Milvus**: 工业级首选, 适合对性能和扩展性要求极高的大规模场景。
-  **Pinecone**: 企业级托管方案, 追求极致开发效率, 适合快速上线与SLA保障。
-  **FAISS**: 算法研究利器, 适合对检索速度有极致要求的离线或小规模场景。
-  **Weaviate**: 知识图谱创新者, 适合需要语义理解和复杂数据关联的场景。
-  **Qdrant**: 性能与成本平衡者, 适合延迟敏感且注重成本控制的生产环境。
-  **Chroma**: 轻量级开源向量数据库的典范, 专为快速原型开发和中小规模RAG场景设计, 极致简化开发流程。

场景化选型建议

-  **快速原型 & 初创团队**: 首选 Pinecone (全托管) 或 Chroma (轻量开源), 降低初期成本。
-  **大规模生产 & 高并发**: 首选 Milvus (分布式) 或 Qdrant (高性能), 保障稳定性。
-  **成本敏感 & 中小数据量**: 首选 Qdrant/Weaviate 开源版, 或利用 pgvector 扩展PG。
-  **混合搜索 & 知识图谱**: 首选 Weaviate, 原生支持向量与结构化数据的混合查询。
-  **算法研究 & 极致性能**: 首选 FAISS, 可深入定制索引算法, 满足科研需求。

向量存储

01 导入依赖库

```
import chromadb
from chromadb.utils import embedding_functions
```

02. 初始化并持久化

```
client = chromadb.PersistentClient(path="./vector_db") # 初始化持久化客户端
collection = client.create_collection() # 创建集合
collection.add() # 添加文档到集合
```

03. 验证与检索测试

```
# 验证文档数量
count = collection.count()
# 测试相似度搜索
results = collection.query(query_texts=[query], n_results=3)
```

LLM 组件

LangChain应用的“大脑”，负责理解自然语言问题并生成精准回答。



云端 LLM (通义千问)



核心特点：
模型强大且稳定，接入简单易用；需 API 密钥和网络环境。



适用场景：
快速原型开发、验证概念、对响应速度要求高的业务。



本地 LLM (Ollama)



核心特点：
支持私有化部署，数据不出域更安全；无网络依赖，但对硬件要求较高。



适用场景：
企业内部部署、对数据隐私有严格合规要求的场景。

LLM 组件

配置 大语言模型 (云端)

1. 在 .env 文件中添加 API 密钥:

```
DASHSCOPE_API_KEY="your-api-key"
```

2. 代码初始化 (指定模型与温度):

```
from langchain_community.llms import Tongyi
llm = Tongyi(
    .....
)
response = llm.invoke(full_prompt)
```

配置本地 LLM

1. 使用Ollama部署本地LLM:

下载Ollama上的大模型到本地

2. 初始化 (需提前下载模型文件):

```
from langchain_community.llms import Ollama
llm = Ollama()
response = llm.invoke(full_prompt)
```


检索组件 (Retrievers)

核心作用：精准检索

从向量存储中，根据用户问题的语义，快速检索出最相关的文档片段。

技术原理：向量相似度

将用户问题向量化，在向量库中进行相似度搜索，返回 Top-K 个最相似文档。

核心价值：消除幻觉

为 LLM 提供精准上下文，有效解决知识过时和“幻觉”问题，确保回答可靠。



检索组件



初始化Chroma向量存储

```
vector_store = Chroma(collection_name="admin_office ",  
embedding_function=embeddings,persist_directory= "./vector_db" )
```



执行检索

```
retriever = vector_store.as_retriever(search_kwargs={ "k" : k}) # 常见检索对象  
docs = retriever.invoke(query) # 直接检索  
results = vector_store.similarity_search_with_score(query=query, k=k) # 返回K个相似度最高的结果
```



验证结果

```
for i, (doc, score) in enumerate(results):  
    print(f"结果 {i+1}: 相似度={score:.4f}")  
    print(f" 内容: {doc.page_content}")
```

什么是LangChain的链(Chain)?

定义

将LLM、提示词模板等多个独立组件组合起来，形成连贯工作流程的机制。

功能

消除组件间繁琐的手动衔接代码，让复杂的AI应用开发变得模块化、标准化且高度可复用。

本质

本质是一个封装了多步骤逻辑的“管道”，数据从一端进入，经过处理后从另一端输出。



总结：链（Chain）是LangChain的“胶水”，它串联组件、封装逻辑，实现了从单一功能调用到复杂流程自动化的跨越。

SequentialChain 链

作用：线性流水线

将多个链按顺序连接，前一个链的输出直接作为后一个链的输入，形成自动化的处理流程。

适用场景：多步骤任务

如：问答 → 总结、翻译 → 润色、摘要 → 关键词提取等。

特点

- 配置简单：只需按顺序传入子链列表即可。
- 流程清晰：支持Verbose模式打印执行细节，便于调试。
- 模块化：复杂任务拆解为独立子任务，提升可维护性。

执行流程可视化



Chain 1: 任务A (输入 → 中间结果)



Chain 2: 任务B (中间结果 → 最终结果)



输出最终结果 (End)

企业文档问答系统代码详解

项目概述：基于RAG的智能问答平台

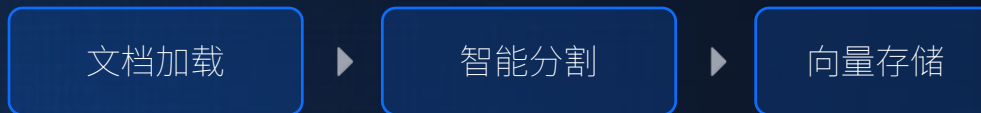
结合LangChain与ChromaDB，将企业非结构化文档转化为可交互的知识系统。通过向量化检索与大模型生成，解决知识获取效率低的痛点，实现精准、可追溯的问答体验。

核心功能特点

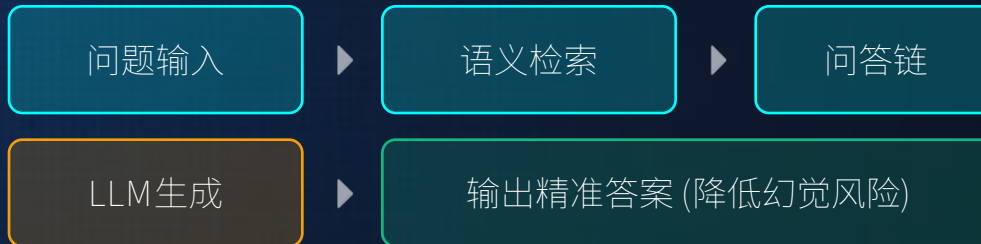
- 多格式加载与智能分割：支持PDF/Word，语义级切片
- 高效向量检索：ChromaDB支持毫秒级相似度匹配
- 增强型问答：集成LLM，基于上下文生成降低幻觉
- 模块化设计：组件解耦，易于扩展与技术迭代

系统架构流程

01 数据准备 Pipeline



02 问答交互 Workflow



架构亮点：

将向量数据库的“检索能力”与大模型的“生成能力”深度融合，确保回答的准确性与可追溯性。

LangChain 核心组件 (二)

深入探讨LangChain中用于流程管控、对话记忆和信息整合的组件。



Middleware (中间件)

实现请求与响应的预处理与后处理，是构建可控AI流程的关键节点。



Memory (记忆)

负责管理和存储对话历史，赋予模型多轮对话的上下文理解能力。



Context (上下文)

整合多维度的外部信息与历史记录，为LLM提供完整的决策依据。

Middleware (中间件) - 流程管控的利器

核心概念：流程中的“过滤器”

Middleware 是在 LLM 请求/响应流程中插入的可插拔组件。它像拦截器一样，在数据进出模型前后执行特定操作。

三大核心作用

 预处理：清洗、格式化、验证用户输入数据

 后处理：格式化、过滤、翻译模型的输出结果

 功能扩展：添加日志记录、速率限制、缓存与重试机制

典型应用场景



输入验证 (Input Validation)

检查用户输入是否合规，防止注入攻击



日志记录 (Logging)

记录请求和响应详情，便于审计与调试



性能监控 (Performance Monitoring)

追踪响应时间，分析系统瓶颈



错误追踪 (Error Tracing)

捕获并记录异常，实现自动重试或降级

Middleware: 日志中间件

日志记录中间件 (Logging)

```
from langchain.agents.middleware import
AgentMiddleware
class LoggingAgentMiddleware(AgentMiddleware):
    log() : 记录结构化日志到文件和控制台-
    on_agent_start() : 代理开始执行时记录-
    on_agent_action() : 代理执行工具动作时记录-
    on_agent_finish() : 代理完成执行时记录
```

执行流程

- 初始化中间件：创建所需的中间件实例
- 创建模拟 Agent：将中间件传递给 MockAgent
- 执行代理：调用 run() 方法执行代理
- 中间件回调：
 - on_agent_start()：代理开始时被调用
 - on_agent_action()：代理执行工具时被调用
 - on_agent_finish()：代理完成时被调用
- 获取结果：获取代理执行结果和中间件产生的数据

LangChain MiddleWare: Human in the Loop (人机协同)



核心定义：打破纯机器闭环，结合人类智慧

在AI自动化工作流中插入人工决策、审核或确认环节，旨在规避AI幻觉与风险，结合AI效率与人类专业判断，提升结果可靠性，是实现工业级AI应用的关键。



风险管控

在敏感操作前加入人工审核，有效拦截AI生成的违规内容，避免执行不当操作，确保业务安全。



提升准确性

利用人类领域知识修正AI幻觉，补全缺失信息，使最终输出结果更贴合真实业务场景与需求。



灵活干预

支持人类在流程中主动中断、修改、驳回或继续AI操作，实现从被动接受到主动控制的转变。



可解释性与溯源

人工介入节点形成完整的操作记录，使AI决策流程透明化、可追溯、可审计，满足企业合规要求。

HumanInTheLoopMiddleware



LangChain 内置的中间件，在 Agent 执行工具调用前插入人工审批环节，实现“人类在循环中”的监督机制，确保关键决策的安全性与合规性。



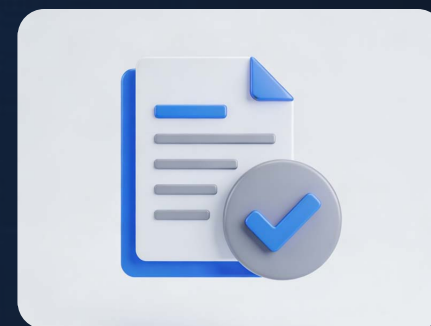
核心功能特性

- 拦截监听工具调用事件
- 支持批准/编辑/拒绝决策
- LangGraph 状态持久化



安全与风险控制

- 防止未授权操作执行
- 人工专家把控关键决策
- 规避潜在的系统风险



合规与审计留痕

- 满足金融医疗行业监管
- 操作审批全流程留痕
- 提升系统整体可信度

工作流程与核心机制

🔄 执行生命周期 (5步)

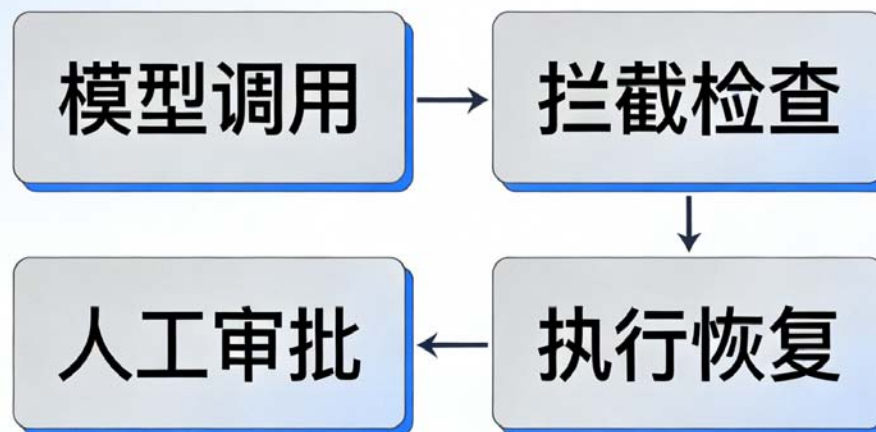
- 🔧 1. 模型调用后拦截：触发 `after_model` 钩子，拦截工具执行前的请求。
- 🛡️ 2. 检查审批策略：基于 `interrupt_on` 配置，判定是否需要人工介入。
- ⚙️ 3. 中断与状态保存：生成中断请求，将 Agent 状态持久化至 Checkpointer。
- 👤 4. 人工决策交互：前端展示待办，等待批准、编辑或拒绝指令。
- 🔄 5. 执行决策与恢复：根据人工反馈恢复流程，继续执行后续任务。

⚙️ 核心技术机制

- 🗄️ LangGraph 状态管理：确保断点续跑的可靠性。
- ⚙️ 细粒度策略控制：支持按工具类型灵活配置规则。

🔗 工作流程示意图

工作流程



核心配置参数详解

⚙️ interrupt_on (Dict, 必填) - 拦截策略配置

- 🔑 Key (工具名称): 如 send_email, execute_sql 等需要控制的工具标识。
- 📁 Value (策略值): True(默认拦截) / False(自动放行/白名单) / InterruptOnConfig(细粒度控制)。

⚙️ InterruptOnConfig 对象 - 细粒度控制参数

- ✓ allowed_decisions: 指定允许的决策类型列表，如 ["approve", "reject"]。
- ☰ description: 自定义中断描述，用于前端展示给审核人员。
- 📄 description_prefix: 全局描述前缀，默认值为 "Tool execution requires approval"。

配置示例与策略设计

基础配置示例 (Python)

```
HumanInTheLoopMiddleware(  
    interrupt_on={  
        "send_email": True, # 需审批，允许所有决策  
        "delete_data": {"allowed_decisions": ["approve", "reject"]}, # 仅允许批准/拒绝  
        "read_data": False # 自动放行  
    }, description_prefix="⚠️ 敏感操作待审批")
```



风险分级策略

高风险操作（如删除数据）严格审批，低风险操作（如查询）自动放行，平衡安全与效率。



权限最小化

根据操作类型限制允许的决策范围，例如禁止编辑SQL语句，仅保留批准或拒绝选项。



用户体验优化

提供清晰的审批提示信息（如前缀标识），帮助人工审核人员快速理解上下文并做出决策。

Memory (记忆) - 实现多轮对话



核心价值：赋予LLM “记忆” 能力

解决LLM无状态问题，存储并管理对话历史，将上下文格式化后作为Prompt传递，实现连贯的多轮对话。



ConversationBufferMemory

完整记录所有对话内容，适用于短对话和需要完整历史的场景。



ConversationSummaryMemory

生成对话摘要压缩历史，适用于长对话和需要提取关键点的场景。



ConversationBufferWindowMemory

只保留最近N条消息，适用于长对话和需要控制上下文长度的场景。



ConversationSummaryBufferMemory

平衡保留最近消息和摘要历史，适用于需要平衡上下文完整性和长度的场景。

Memory (记忆) - 实现多轮对话



ConversationTokenBufferMemory

基于token数量管理上下文，适用于对模型token限制敏感的场景。



ConversationKGMemory

提取和管理对话中的实体关系，适用于需要构建知识结构的场景。



BasicMemory

基础对话内存管理，支持多种内存类型，适用于单会话应用和快速开发测试。



SessionMemory

多会话内存管理，支持多种存储类型，适用于多用户应用和需要会话持久化的场景。



AdvancedMemory

高级内存管理，集成所有内存类型，适用于复杂对话管理需求的应用。



VectorStoreRetrieverMemory

基于向量相似度检索相关记忆片段，而非简单的时间窗口，适合需要高相关性的复杂场景。

Memory：使用示例与类型对比

基础使用示例 (ConversationBufferMemory)

```
# 创建基础内存
buffer_memory = ConversationBufferMemory()
# 添加对话
buffer_memory.chat_memory.add_user_message("你好，我是张三")
buffer_memory.chat_memory.add_ai_message("你好张三，有什么我可以帮你的吗？")
# 获取对话历史
for message in buffer_memory.chat_memory.messages:
    if isinstance(message, HumanMessage):
        print(f"human: {message.content}")
    elif isinstance(message, AIMessage):
        print(f"ai: {message.content}")
# 获取内存变量
memory_vars = buffer_memory.load_memory_variables({})
```

Memory 类型选择原则



短对话场景

推荐：ConversationBufferMemory (完整存储历史)



长对话/节省Token

推荐：ConversationSummaryMemory (存储摘要)



性能优先/固定窗口

推荐：ConversationBufferWindowMemory (只存最近k轮)



高相关性/海量历史

推荐：VectorStoreRetrieverMemory (向量检索)

Context (上下文) - 整合多维度信息

核心概念定义

Context 是整合所有与当前请求相关信息的核心，为 LLM 提供完整的决策依据。

Context $\neq$ Memory

Memory 是 Context 的子集。Context 范围更广，不仅包含对话历史，还融合了环境、知识等多源信息。



对话上下文

来自 Memory 的历史对话记录，维持会话连贯性。



环境上下文

用户画像、系统配置、当前时间地点等元数据。



知识上下文

从知识库、文档中实时检索到的相关信息。



核心挑战



Token 限制与成本控制



信息新鲜度维护



信息相关性筛选



多源异构信息融合

Context: 构建多维度上下文

📄 实现原理：结构化 Prompt 模板



模板定义 (ChatPromptTemplate)

整合 System 提示词、用户信息、对话历史占位符及用户输入，形成标准化交互流程。



动态上下文注入

构建包含用户画像、实时时间等环境信息的字典，在调用时动态传递给 LLM。

📄 核心代码示例 (Python)

```
# 自定义上下文管理器
class MultiDimensionalContext: .....

# 创建上下文管理器
context_manager = MultiDimensionalContext()

# 构建上下文函数
def build_context(inputs: Dict[str, Any]) -> Dict[str, Any]: .....

# 创建提示模板
prompt = ChatPromptTemplate.from_template(.....)

# 创建Chain
chain = RunnableSequence(
    RunnableLambda(build_context),
    prompt,
    llm,
    StrOutputParser()
)

# 运行Chain
result = chain.invoke(test_input)
```

三大组件协同工作机制



实战案例：整合三大组件构建多轮对话系统

核心业务需求

-  多轮对话支持
记住用户的历史问题和偏好，提供连贯体验
-  输入验证与安全
实时过滤敏感内容，确保对话合规性
-  上下文整合
融合用户画像、时间、地点等多维环境信息
-  全链路日志记录
详细记录交互日志，便于后续分析与调试

技术架构实现

-  Middleware 中间件层
使用 Logging 与 InputValidation 组件处理请求
-  Memory 记忆管理
采用 BufferWindowMemory 限制最近5轮对话
-  Context 上下文管理
ContextManager 整合用户信息与环境变量
-  Core 核心链
通过 ConversationChain 串联所有组件形成闭环

The background is a dark blue field filled with a complex network of glowing, thin blue lines that intersect and curve, creating a sense of depth and movement. Scattered throughout the scene are various sequences of white and light blue binary code (0s and 1s), some appearing as horizontal streams and others as small clusters, contributing to a high-tech, digital aesthetic.

感谢聆听

Q & A